**Prakash Thapa**
**D20A- 32**

## Experiment No.: 2

## Create a Blockchain using Python

1. **Aim**

To create a Blockchain using Python with transaction support and demonstrate its working using Flask and Postman.

2. **Objectives**

- Implement a basic Blockchain using Python.
- Add transaction functionality.
- Mine a block only when transactions are available.
- Implement Proof of Work with **000 leading zeros**.
- Validate the Blockchain using Flask APIs.

3. **Requirements**

- Python
- Flask
- VS Code
- Postman

**4. Implementation Logic**

Blockchain Class

The following functions are implemented:

- __init__() – initializes the chain and transaction list and creates the Genesis block.
- create_block() – creates and adds a block to the chain.
- get_previous_block() – returns the last block.
- create_Transactions() – adds a new transaction to the pending transaction list.
- proof_of_work() – searches for a proof whose hash contains **000 leading zeros**.
- hash() – generates the SHA-256 hash of a block.
- is_chain_valid() – verifies the Blockchain and Proof of Work.

**Mining Condition**

A block is mined only when pending transactions are available.

After the transactions are included in the newly mined block, the pending transaction list is cleared.

**5. Proof of Work**

The cryptographic puzzle used is:

```
hash_operation = hashlib.sha256(
   str(new_proof**2 - previous_proof**2).encode()
).hexdigest()

if hash_operation[:3] == '000':
   check_proof = True
```

Thus, the generated hash must begin with:

000

**6. Flask API Endpoints**

| Method | Endpoint | Function |
|---|---|---|
| POST | /add_transaction | Adds a transaction |
| GET | /mine_block | Mines a block |
| GET | /get_chain | Displays the Blockchain |
| GET | /is_valid | Validates the Blockchain |

**7. Postman Testing**

The following operations were successfully tested:

1. GET /get_chain – displayed the Genesis block.
2. GET /mine_block without transactions – returned **"No transactions to mine."**
3. POST /add_transaction – successfully added a transaction.

4. GET /mine_block – successfully mined a block containing the transaction.
5. GET /get_chain – displayed the updated Blockchain.
6. GET /is_valid – confirmed that the Blockchain is valid.

Sample Transaction

```
{
  "sender": "Alice",
  "receiver": "Bob",
  "amount": 100
}
```

**Implementation Steps**

1. Create a `Blockchain` class and initialize an empty `chain` and `transactions` list.
2. Create the **Genesis block** using `create_block()` with:
   - `proof = 1`
   - `previous_hash = '0'`
3. Implement `create_block()` to create a block containing:
   - Index
   - Timestamp
   - Proof
   - Previous hash
   - Transactions
4. Implement `get_previous_block()` to retrieve the latest block.
5. Implement `create_Transactions()` to add sender, receiver, and amount to the pending transaction list.
6. Modify `proof_of_work()` to find a proof whose SHA-256 hash has **`000` leading zeros**.
7. Implement `hash()` to generate the SHA-256 hash of a block.
8. Implement `is_chain_valid()` to verify the previous hash and Proof of Work of each block.
9. Create a Flask WebApp and initialize the Blockchain object.
10. Create Flask routes:

- `/add_transaction`
- `/mine_block`
- `/get_chain`
- `/is_valid`

11. In `/mine_block`, check whether pending transactions are available. If no transactions exist, return **"No transactions to mine."**

12. If transactions exist, perform Proof of Work, create the new block, and clear the pending transactions.
13. Run the Flask application on **port 5000**.
14. Use **Postman** to add transactions, mine blocks, display the chain, and verify Blockchain validity.
15. Confirm that `/is_valid` returns **"All good. The Blockchain is valid."**

### 8. Result

The Blockchain was successfully implemented using Python with transaction support and Proof of Work requiring **000 leading zeros**. The Flask APIs were successfully tested using Postman, and the Blockchain was verified as valid.

### Colab Code Snippets:

1.
```
import datetime
import hashlib
import json
```

2.
```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_block(proof = 1, previous_hash = '0')

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash}
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
```

```
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
        return True
```

3.

```
# Creating a Blockchain
blockchain = Blockchain()
```

4.

```
# Mining a new block
def mine_block():
```

```
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash']}
    return response
```

5.

```
# Getting the full Blockchain
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return response
```

6.

```
# Checking if the Blockchain is valid
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain
is not valid.'}
    return response
```

7.

```
print("Functions Menu:")
print("===============")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
```

```
  print(mine_block())
elif choice == 2:
  print(get_chain())
elif choice == 3:
  print(is_valid())
else :
  print("Invalid choice")
```

```
Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :1
{'message': 'Congratulations, you just mined a block!', 'index': 2,
'timestamp': '2026-08-21 09:35:21.556742', 'proof': 533, 'previous_hash':
'd6fd8693d9adbf85a98e79b24b63d57dd4f66ed459b971664d89d89dea7282a3'}
```

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof = 1, previous_hash = '0')

    def create_Transactions(self, sender, receiver, amount):
        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }

        self.transactions.append(transaction)

        return len(self.chain) + 1


blockchain.create_Transactions("Alice", "Bob", 100)
2

blockchain.transactions
```

```
[{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}]


# Mining a new block
def mine_block():

    if len(blockchain.transactions) == 0:
        return {
            'message': 'No transactions to mine.'
        }

    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)

    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(proof, previous_hash)

    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash']
    }

    return response
```

After that run:

```
mine_block()

Output:
{'message': 'Congratulations, you just mined a block!',
 'index': 2,
 'timestamp': '2026-08-21 09:53:40.406033',
 'proof': 533,
 'previous_hash':
'01b97b1b1c7d1a26d79da573d7dcc27e990988e6b05df716214cf320ba2c37f0'}
```

Change in cell 2:

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof = 1, previous_hash = '0', transactions=[])

def create_block(self, proof, previous_hash, transactions):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': transactions}

Run cell 2 and 3


Change cell 4:
# Mining a new block
def mine_block():

    # Mine only if transactions are available
    if len(blockchain.transactions) == 0:
        return {
            'message': 'No transactions to mine.'
        }

    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)

    previous_hash = blockchain.hash(previous_block)

    # Store pending transactions in the block
    transactions = blockchain.transactions.copy()

    block = blockchain.create_block(
        proof,
        previous_hash,
```

```
        transactions
    )

    # Remove transactions after adding them to the block
    blockchain.transactions = []

    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

    return response


blockchain.create_Transactions("Alice", "Bob", 100)
3
```

---

```
blockchain.transactions
[{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}]
```

---

```
mine_block()
{'message': 'Congratulations, you just mined a block!',
 'index': 3,
 'timestamp': '2026-08-21 11:55:53.534515',
 'proof': 45293,
 'previous_hash':
'b84ffd800b6f0d22672e9f21497b223cff21fdb96d379a891b72919ed074aca0',
 'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}]}
```

---

```
blockchain.transactions
[]
```

---

- **Modify the method, proof_of_work() to search for the golden nonce**
- Cryptographic Puzzle is to have **“000” leading zeros** in the Block Hash

```
if hash_operation[:3] == '000':


blockchain.create_Transactions("Alice", "Bob", 100)
2
```

---

```
mine_block()
{'message': 'Congratulations, you just mined a block!',
 'index': 2,
 'timestamp': '2026-08-31 04:38:26.769762',
 'proof': 533,
 'previous_hash':
'1873623428673697637b28e5c0797ca2d40dab006b752e4c56029b72a2347014',
 'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}]}


previous_block = blockchain.get_previous_block()
previous_proof = blockchain.chain[-2]['proof']
proof = previous_block['proof']

hash_operation = hashlib.sha256(
    str(proof**2 - previous_proof**2).encode()
).hexdigest()

print(hash_operation)
print(hash_operation[:3])

0000c00870f23a23ae80377298491b091db400d575be0efbde5b310f2f763ed1
000


print("Functions Menu:")
print("===============")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
```

```
  print(mine_block())
elif choice == 2:
  print(get_chain())
elif choice == 3:
  print(is_valid())
else :
  print("Invalid choice")
```

```
Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :3
{'message': 'All good. The Blockchain is valid.'}
```

```
Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :2
{'chain': [{'index': 1, 'timestamp': '2026-08-31 04:38:23.984663',
'proof': 1, 'previous_hash': '0', 'transactions': []}, {'index': 2,
'timestamp': '2026-08-31 04:38:26.769762', 'proof': 533, 'previous_hash':
'1873623428673697637b28e5c0797ca2d40dab006b752e4c56029b72a2347014',
'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}]}],
'length': 2}
```

**FInal [blockchain.py](blockchain.py):**

```
# Module 1 - Create a Blockchain

# To be installed:
# Flask==0.12.2: pip install Flask==0.12.2
# Postman HTTP Client

# Importing the libraries
import datetime
import hashlib
```

```
import json
from flask import Flask, jsonify, request

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions.copy()
        }

        self.chain.append(block)

        # Remove transactions from pending list
        self.transactions = []

        return block

    def get_previous_block(self):
        return self.chain[-1]

    def create_Transactions(self, sender, receiver, amount):
        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }

        self.transactions.append(transaction)

        return len(self.chain) + 1

    def proof_of_work(self, previous_proof):
        new_proof = 1
```

```
    check_proof = False

    while check_proof is False:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()

        # Cryptographic puzzle: 000 leading zeros
        if hash_operation[:3] == '000':
            check_proof = True
        else:
            new_proof += 1

    return new_proof

def hash(self, block):
    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]

        if block['previous_hash'] != self.hash(previous_block):
            return False

        previous_proof = previous_block['proof']
        proof = block['proof']

        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()

        if hash_operation[:3] != '000':
            return False
```

```
            previous_block = block
            block_index += 1

        return True


# Part 2 - Mining our Blockchain

# Creating a Web App
app = Flask(__name__)

# Creating a Blockchain
blockchain = Blockchain()


# Creating a transaction
@app.route('/add_transaction', methods=['POST'])
def add_transaction():

    json_data = request.get_json()

    sender = json_data['sender']
    receiver = json_data['receiver']
    amount = json_data['amount']

    index = blockchain.create_Transactions(
        sender,
        receiver,
        amount
    )

    response = {
        'message': 'Transaction added successfully.',
        'block_index': index
    }

    return jsonify(response), 201


# Mining a new block
@app.route('/mine_block', methods=['GET'])
def mine_block():

    # Mine only if transactions are available
```

```
    if len(blockchain.transactions) == 0:
        response = {
            'message': 'No transactions to mine.'
        }

        return jsonify(response), 400

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)

    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(
        proof,
        previous_hash
    )

    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

    return jsonify(response), 200


# Getting the full Blockchain
@app.route('/get_chain', methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

    return jsonify(response), 200
```

```
# Checking if the Blockchain is valid
@app.route('/is_valid', methods=['GET'])
def is_valid():

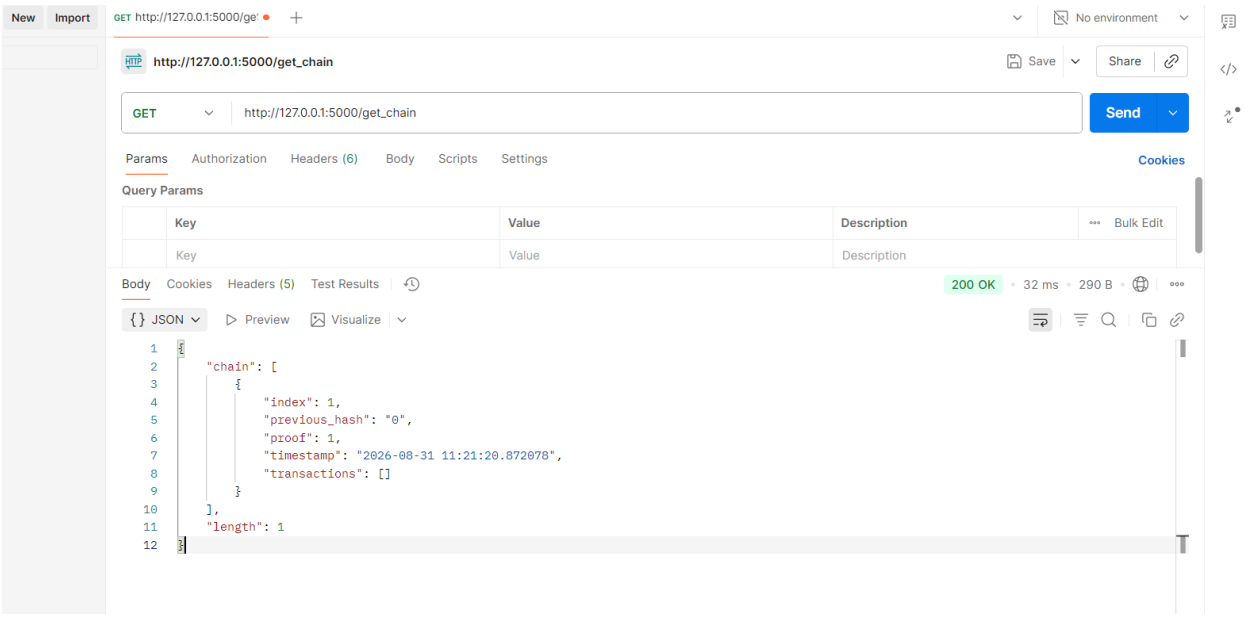
    is_valid = blockchain.is_chain_valid(
        blockchain.chain
    )

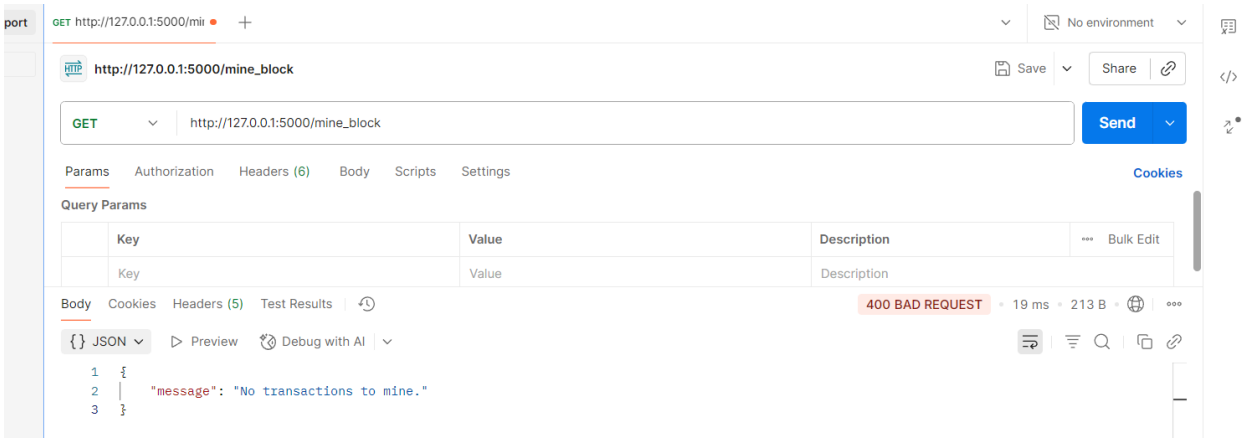
    if is_valid:
        response = {
            'message': 'All good. The Blockchain is valid.'
        }
    else:
        response = {
            'message': 'Houston, we have a problem. '
                       'The Blockchain is not valid.'
        }

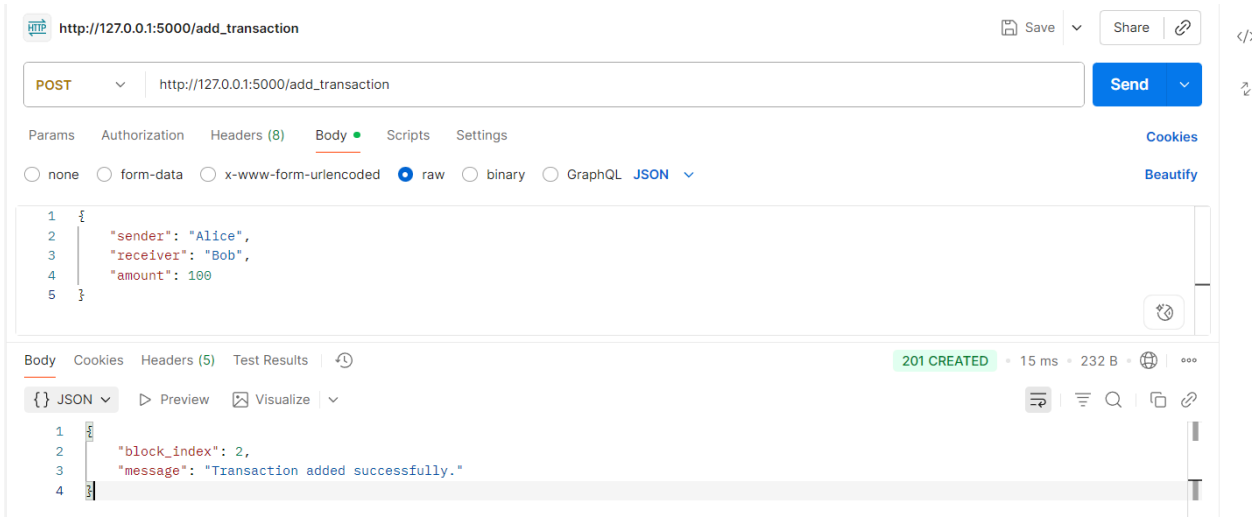
    return jsonify(response), 200

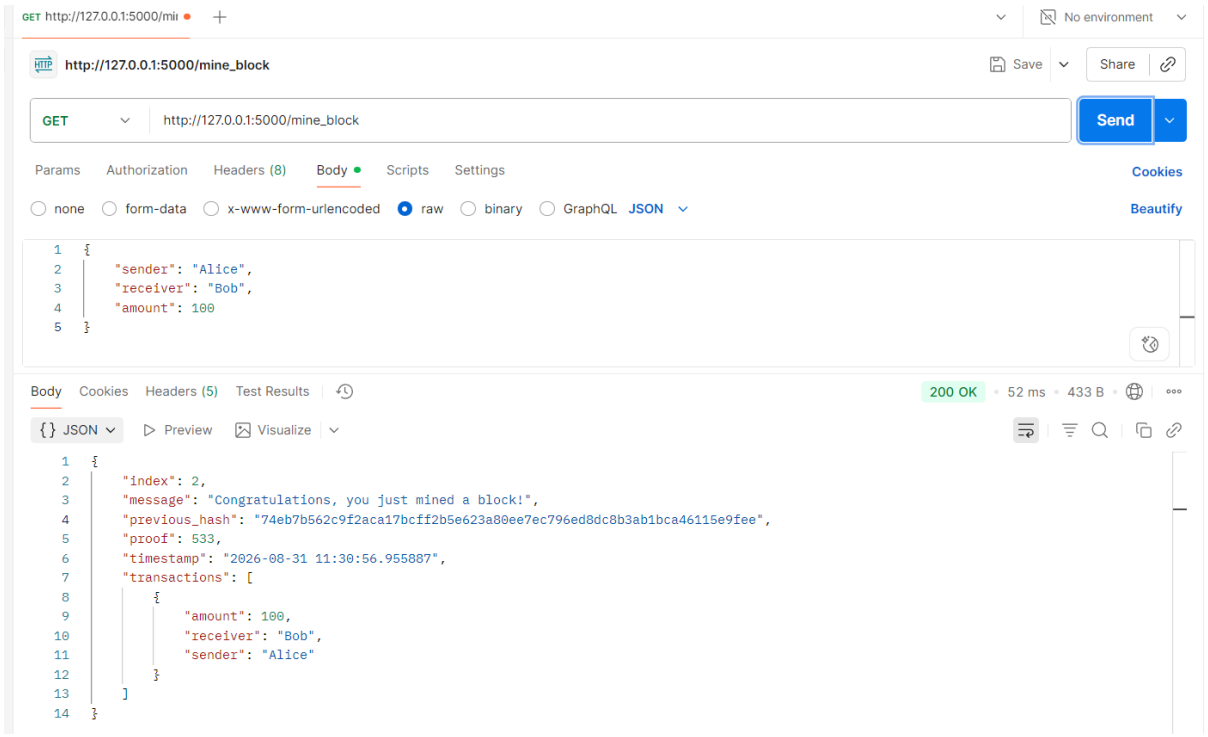

# Running the app
app.run(host='0.0.0.0', port=5000)
```

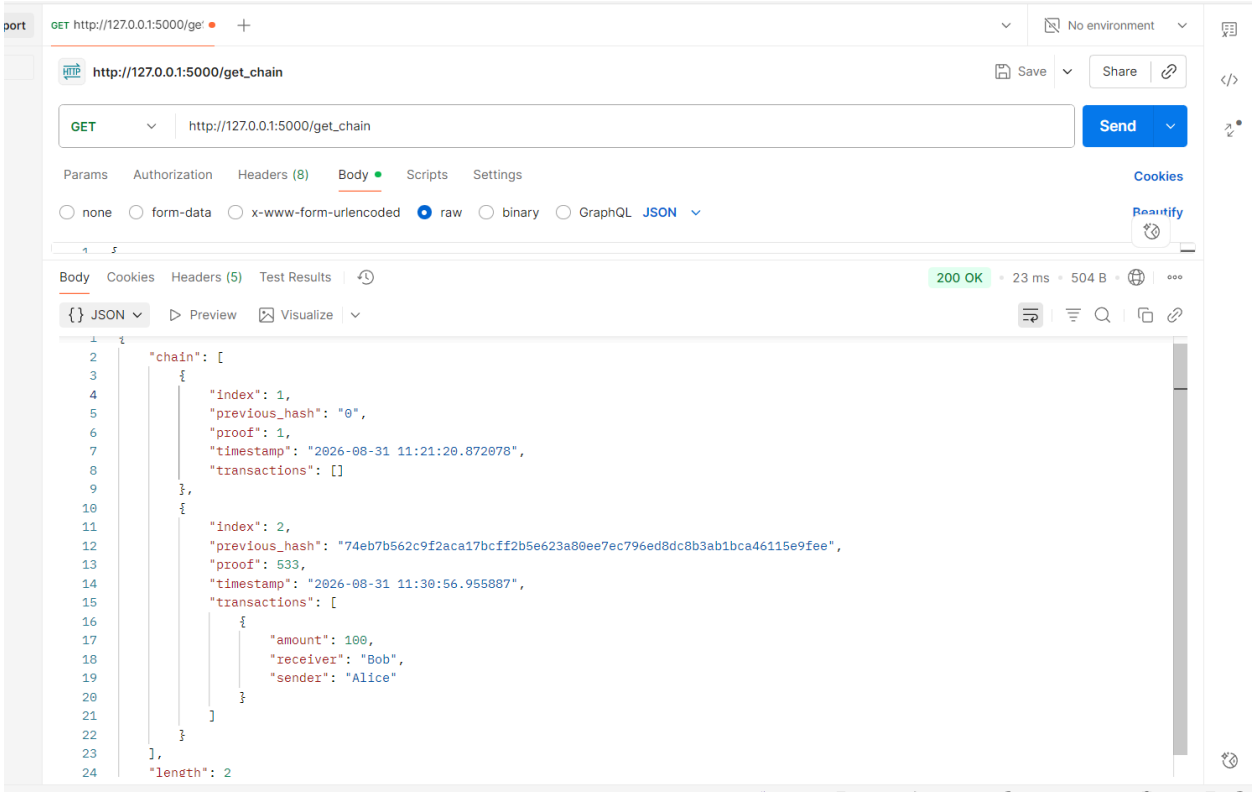
## PostMan Testing:

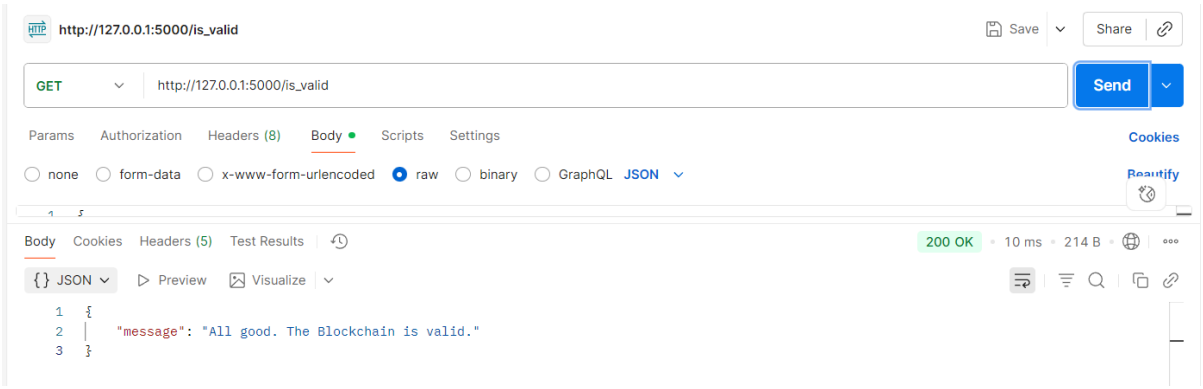



port
GET http://127.0.0.1:5000/mi
No environment
http://127.0.0.1:5000/mine_block
Save
Share
GET
http://127.0.0.1:5000/mine_block
Send
Params
Authorization
Headers (6)
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
400 BAD REQUEST
19 ms
213 B
JSON
Preview
Debug with AI
{
"message": "No transactions to mine."
}


http://127.0.0.1:5000/add_transaction
Save
Share
POST
http://127.0.0.1:5000/add_transaction
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"sender": "Alice",
"receiver": "Bob",
"amount": 100
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
15 ms
232 B
JSON
Preview
Visualize
{
"block_index": 2,
"message": "Transaction added successfully."
}


GET http://127.0.0.1:5000/mi
No environment
http://127.0.0.1:5000/mine_block
Save
Share
GET
http://127.0.0.1:5000/mine_block
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"sender": "Alice",
"receiver": "Bob",
"amount": 100
}
Body
Cookies
Headers (5)
Test Results
200 OK
52 ms
433 B
JSON
Preview
Visualize
{
"index": 2,
"message": "Congratulations, you just mined a block!",
"previous_hash": "74eb7b562c9f2aca17bcff2b5e623a80ee7ec796ed8dc8b3ab1bca46115e9fee",
"proof": 533,
"timestamp": "2026-08-31 11:30:56.955887",
"transactions": [
{
"amount": 100,
"receiver": "Bob",
"sender": "Alice"
}
]
}


GET http://127.0.0.1:5000/ge
No environment
http://127.0.0.1:5000/get_chain
Save
Share
GET
http://127.0.0.1:5000/get_chain
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
23 ms
504 B
JSON
Preview
Visualize
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-31 11:21:20.872078",
"transactions": []
},
{
"index": 2,
"previous_hash": "74eb7b562c9f2aca17bcff2b5e623a80ee7ec796ed8dc8b3ab1bca46115e9fee",
"proof": 533,
"timestamp": "2026-08-31 11:30:56.955887",
"transactions": [
{
"amount": 100,
"receiver": "Bob",
"sender": "Alice"
}
]
}
],
"length": 2


http://127.0.0.1:5000/is_valid
Save
Share
GET
http://127.0.0.1:5000/is_valid
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
10 ms
214 B
{} JSON
Preview
Visualize
{
"message": "All good. The Blockchain is valid."
}